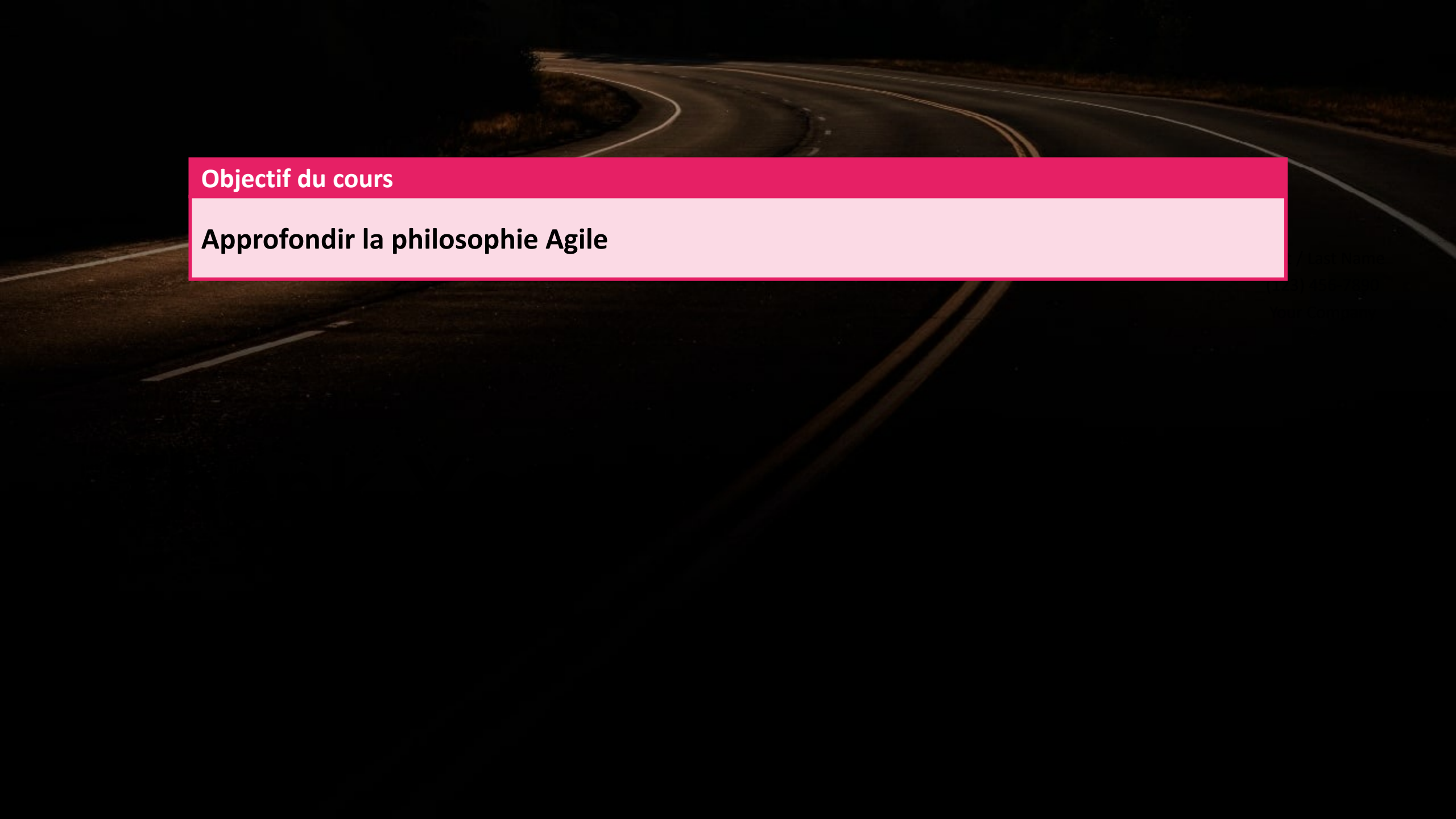


A long, straight asphalt road stretches from the foreground into the distance, vanishing at a horizon line. The road is flanked by dry, yellowish-brown grass and low hills. A dashed white line runs down the center of the road, and solid white lines mark the edges. The sky is dark and overcast, with a few distant mountains visible on the horizon. The overall mood is one of vastness and forward movement.

AGILE

Go deeper

A dark, winding road at night, illuminated by a soft light source, creating a sense of depth and mystery. The road curves to the right, with white dashed lines marking the edge. A pink rectangular box is overlaid on the left side of the road, containing text.

Objectif du cours

Approfondir la philosophie Agile

Définir Agile



Définition

Agile

- C'est une philosophie
- Qui définit un ensemble de valeurs et de principes
- Pour réaliser un produit de haute qualité
- Dans un monde à forte incertitude

(Définition du premier chapitre)

Agile

- adopter une approche itérative et incrémentale
 - menée dans un esprit collaboratif
 - pour générer un produit de haute qualité
 - en tenant compte de l'évolution client
-
- **flexibilité, répétition, ouverture aux changements**

Définition complète

- C'est une philosophie qui définit un ensemble de valeurs et de principes
- Pour réaliser un produit de haute qualité
- En adoptant une approche itérative et incrémentale menée dans un esprit collaboratif
- Et en tenant compte de l'évolution client (monde à forte incertitude)

Philosophie Agile



Le Manifeste Agile

History: The Agile Manifesto

On February 11-13, 2001, at The Lodge at Snowbird ski resort in the Wasatch mountains of Utah, seventeen people met to talk, ski, relax, and try to find common ground—and of course, to eat. What emerged was the Agile ‘Software Development’ Manifesto. Representatives from Extreme Programming, SCRUM, DSDM, Adaptive Software Development, Crystal, Feature-Driven Development, Pragmatic Programming, and others sympathetic to the need for an alternative to documentation driven, heavyweight software development processes convened.

Now, a bigger gathering of organizational anarchists would be hard to find, so what emerged from this meeting was symbolic—a *Manifesto for Agile Software Development*—signed by all participants. The only concern with the term *agile* came from Martin Fowler (a Brit for those who don’t know him) who allowed that most Americans didn’t know how to pronounce the word ‘agile’.

Alistair Cockburn’s initial concerns reflected the early thoughts of many participants. "I personally didn’t expect that this particular group of agilites to ever agree on anything substantive." But his post-meeting feelings were also shared, "Speaking for myself, I am delighted by the final phrasing [of the Manifesto]. I was surprised that the others appeared equally delighted by the final phrasing. So we did agree on something substantive."

Naming ourselves "The Agile Alliance," this group of independent thinkers about software development, and sometimes competitors to each other, agreed on the *Manifesto for Agile Software Development* displayed on the title page of this web site.

But while the Manifesto provides some specific ideas, there is a deeper theme that drives many, but not all, to be sure, members of the alliance. At the close of the two-day meeting, Bob Martin joked that he was about to make a "mushy" statement. But while tinged with humor, few disagreed with Bob’s sentiments—that we all felt privileged to work with a group of people who held a set of compatible values, a set of values based on trust and respect for each other and promoting organizational models based on people, collaboration, and building the types of organizational communities in which we would want to work. At the core, I believe Agile Methodologists are really about "mushy" stuff—about delivering good products to customers by operating in an environment that does more than talk about "people as our most important asset" but actually "acts" as if people were the most important, and lose the word "asset". So in the final analysis, the meteoric rise of interest in—and sometimes tremendous criticism of—Agile Methodologies is about the mushy stuff of values and culture.

For example, I think that ultimately, Extreme Programming has mushroomed in use and interest, not because of pair-programming or refactoring, but because, taken as a whole, the practices define a developer community freed from the baggage of Dilbertesque corporations. Kent Beck tells the story of an early job in which he estimated a programming effort of six weeks for two people. After his manager reassigned the other programmer at the beginning of the project, he completed the project in twelve weeks—and felt terrible about himself! The boss—of course—harangued Kent about how slow he was throughout the second six weeks. Kent, somewhat despondent because he was such a "failure" as a programmer, finally realized that his original estimate of 6 weeks was extremely accurate—for 2 people—and that his "failure" was really the manager’s failure , indeed, the failure of the standard "fixed" process mindset that so frequently plagues our industry.

This type of situation goes on every day—marketing, or management, or external customers, internal customers, and, yes, even developers—don’t want to make hard trade-off decisions, so they impose irrational demands through the imposition of corporate power structures. This isn’t merely a software development problem, it runs throughout Dilbertesque organizations.

In order to succeed in the new economy, to move aggressively into the era of e-business, e-commerce, and the web, companies have to rid themselves of their Dilbert manifestations of make-work and arcane policies. This freedom from the inanities of corporate life attracts proponents of Agile Methodologies, and scares the bejeebers (you can’t use the word ‘shit’ in a professional paper) out of traditionalists. Quite frankly, the Agile approaches scare corporate bureaucrats— at least those that are happy pushing process for process’ sake versus trying to do the best for the "customer" and deliver something timely and tangible and "as promised"—because they run out of places to hide.

The Agile movement is not anti-methodology, in fact, many of us want to restore credibility to the word methodology. We want to restore a balance. We embrace modeling, but not in order to file some diagram in a dusty corporate repository. We embrace documentation, but not hundreds of pages of never-maintained and rarely-used tomes. We plan, but recognize the limits of planning in a turbulent environment. Those who would brand proponents of XP or SCRUM or any of the other Agile Methodologies as "hackers" are ignorant of both the methodologies and the original definition of the term hacker.

The meeting at Snowbird was incubated at an earlier get together of Extreme Programming proponents, and a few "outsiders," organized by Kent Beck at the Rogue River Lodge in Oregon in the spring of 2000. At the Rogue River meeting attendees voiced support for a variety of "Light" methodologies, but nothing formal occurred. During 2000 a number of articles were written that referenced the category of "Light" or "Lightweight" processes. A number these articles referred to "Light methodologies, such as Extreme Programming, Adaptive Software Development, Crystal, and SCRUM". In conversations, no one really liked the moniker "Light", but it seemed to stick for the time being.

In September 2000, Bob Martin from Object Mentor in Chicago, started the next meeting ball rolling with an email; "I'd like to convene a small (two day) conference in the January to February 2001 timeframe here in Chicago. The purpose of this conference is to get all the lightweight method leaders in one room. All of you are invited; and I'd be interested to know who else I should approach." Bob set up a Wiki site and the discussions raged.

Early on, Alistair Cockburn weighed in with an epistle that identified the general disgruntlement with the word ‘Light’: "I don't mind the methodology being called light in weight, but I'm not sure I want to be referred to as a lightweight attending a lightweight methodologists meeting. It somehow sounds like a bunch of skinny, feebleminded lightweight people trying to remember what day it is."

The fiercest debate was over location! There was serious concern about Chicago in wintertime—cold and nothing fun to do; Snowbird, Utah—cold, but fun things to do, at least for those who ski on their heads like Martin Fowler tried on day one; and Anguilla in the Caribbean—warm and fun, but time consuming to get to. In the end, Snowbird and skiing won out; however, a few people—like Ron Jeffries—want a warmer place next time.

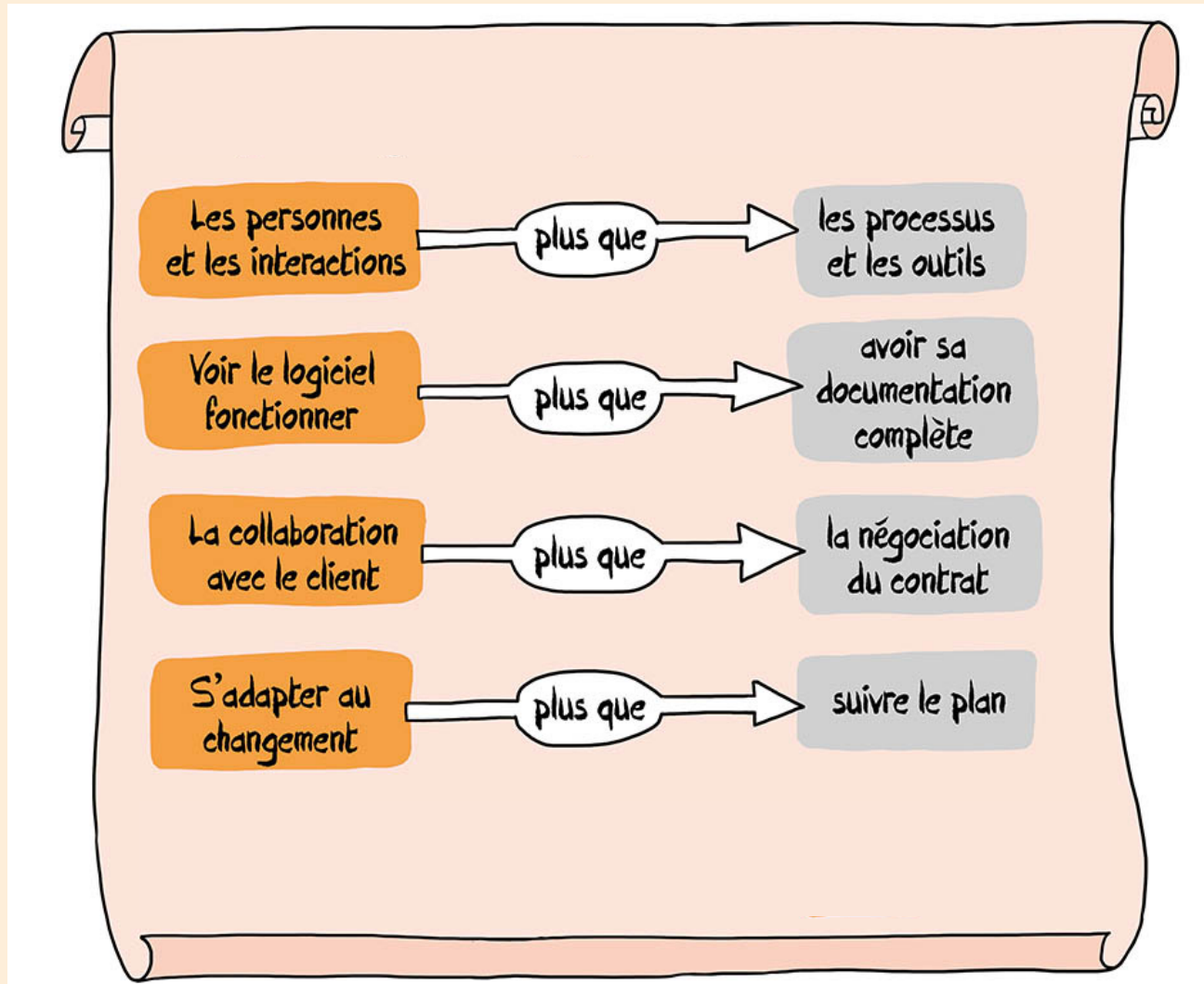
We hope that our work together as the Agile Alliance helps others in our profession to think about software development, methodologies, and organizations, in new- more agile - ways. If so, we’ve accomplished our goals.

Jim Highsmith, for the Agile Alliance

©2001 Jim Highsmith

<http://agilemanifesto.org/history.html>

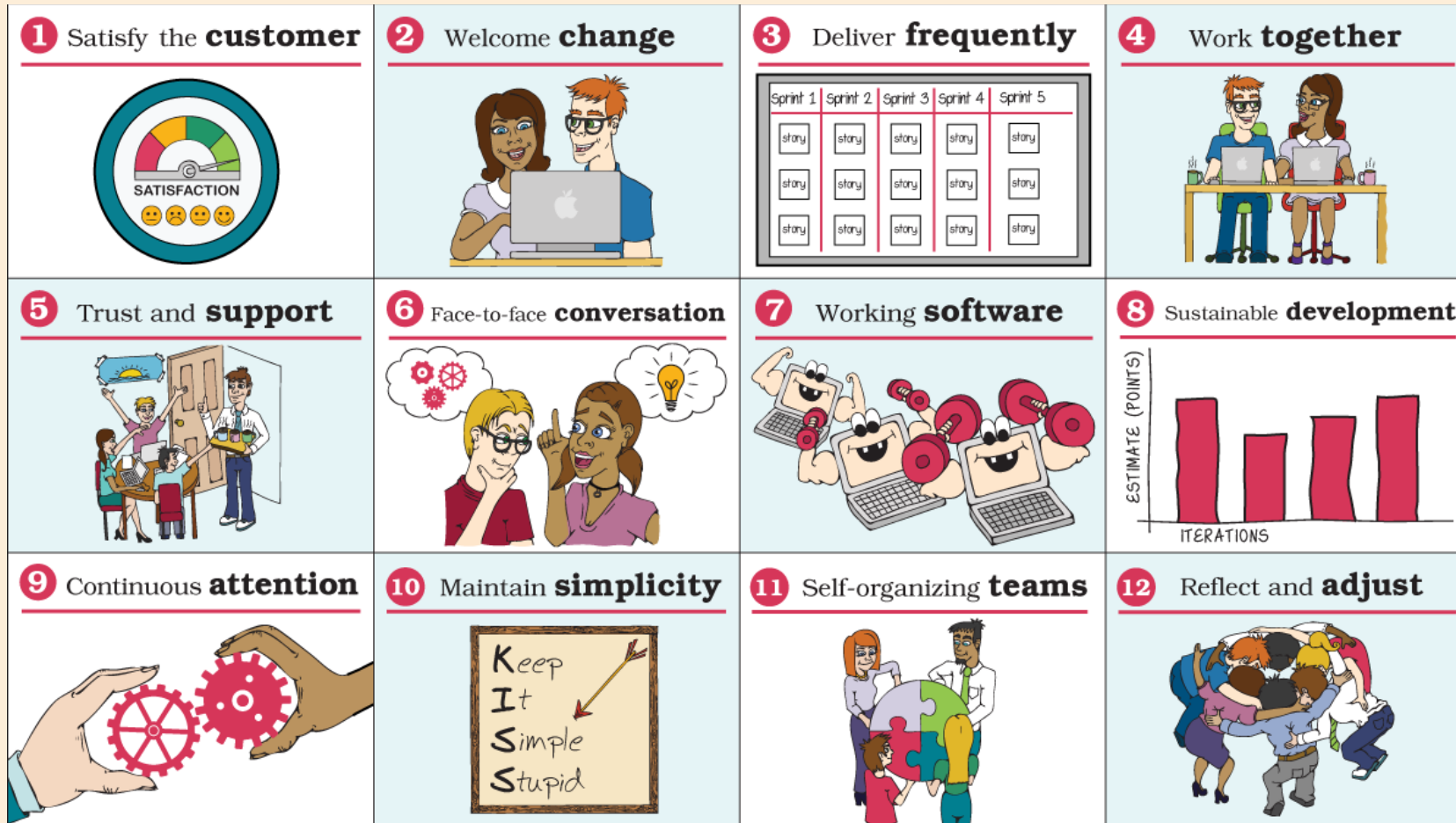
Manifeste Agile : les 4 valeurs



Question :

Expliquez en détail chaque valeur

Manifeste Agile : les 12 principes



Déclaration des droits

Affirmation

Agile n'est pas simplement un ensemble de règles. Bien au contraire, Agile est un ensemble de droits, d'attentes et de disciplines formant la plateforme conceptuelle d'une profession éthique

Question :

Pourquoi la déclaration des droits ?

- Kent Beck avait déclaré que le but de Agile **était d'abord de réparer la fracture entre l'entreprise et les développeurs**, d'où l'apparition des droits suivants
 - droits pour le client
 - droits pour le développeurs

Les droits

Question :

A votre avis, quels sont ces droits ?

Les clients/managers ont le droit

- d'avoir un plan d'ensemble : qu'est-ce qui peut être accompli, quand et à quel prix ?
- d'obtenir le plus de valeur après chaque itération
- de voir les progrès réalisés sur l'avancement du logiciel
- d'être informés des changements de calendrier et d'estimation.
- d'annuler à tout moment et de rester avec un système fonctionnel.

Les développeurs ont le droit ...

- de savoir ce que l'on attend d'eux et quelles sont les priorités.
- de produire du code de haute qualité.
- de demander et recevoir de l'aide.
- de mettre à jour leurs estimations.
- d'accepter des responsabilités au lieu de se les voir infligés.

Droits client #1

Customers have the right to an overall plan and to know what can be accomplished when and at what cost.

Many people have claimed that up-front planning is not part of Agile development. The very first customer right belies that claim. **Of course the business needs a plan.** Of course that plan must include schedule and cost. And, of course that plan should be as accurate and precise as practical.

In short, **we cannot agree to deliver fixed scopes on hard dates.** Either the scopes or the dates must be soft.

Customers have the right to this kind of probability-based plan **because they cannot manage their business without it.**

Droits client #2

Customers have the right to get the most possible value out of every iteration.

Agile breaks up the development effort into fixed time boxes called *iterations*. The business has the right to expect that the developers will work on the most important things at any given time, and that each iteration will provide them the maximum possible *usable business value*. This priority of value is specified by the customer during the planning sessions at the beginning of each iteration. The customers choose the stories that give them the highest return on investment and that can fit within the developer's estimation for the iteration.

Droits client #5

Customers have the right to be informed of schedule and estimate changes in time to choose how to alter the scope to meet the required date.

Customers may cancel at any time and be left with a useful working system reflecting investment to date.

Note that customers do not have the right to demand conformance to the schedule. Their right is limited to managing the schedule by changing the scope. The most important thing this right confers is the right to *know* that the schedule is in jeopardy so that it can be managed in a timely fashion.

Droits développeur #2

Developers have the right to produce high-quality work at all times.

This may be the most profound of all these rights. Developers have the right to do good work. The business has no right to tell developers to cut corners or do low-quality work. Or, to say this differently, the business has no right to force developers to ruin their professional reputations or violate their professional ethics.

Droits développeur #4

Developers have the right to make and update their own estimates.

No one can estimate a task for you. And if you estimate a task, you can always change your estimate when new factors come to light. Estimates are guesses. They are intelligent guesses to be sure, but they're still guesses. They are guesses that get better with time. Estimates are never commitments.

Estimation != Engagement

Droits développeur #5

Developers have the right to accept their responsibilities instead of having them assigned.

Professionals *accept* work, they are not assigned work. A professional developer has every *right to say “no”* to a particular job or task. It may be that the developer does not feel confident in their ability to complete the task, or it may be that the developer believes the task better suited for someone else. Or, it may be that the developer rejects the tasks for personal or moral reasons.⁶

In any case, *the right to accept comes with a cost*. Acceptance implies responsibility. The *accepting developer becomes responsible for the quality and execution of the task, for continually updating the estimate* so that the schedule can be managed, for communicating status to the whole team, and for asking for help when help is needed.

« Conclusion »

- Agile n'est ni un processus ni un engouement passager
- Agile n'est pas simplement un ensemble de règles
- Agile est un ensemble
 - De droits
 - D'attentes
 - Et de discipline
- Formant la plateforme conceptuelle d'une profession éthique

Agile is not a process. Agile is not a fad. Agile is not merely a set of rules. Rather, Agile is a set of rights, expectations, and disciplines of the kind that form the basis of an ethical profession.

**Conséquence de
cette philosophie**

Accepter l'incertitude

« Accepter une réalité et comprendre que dans le développement logiciel tout n'est pas prévisible »

Accepter le changement

« Accepter le changement à bras ouverts plutôt que de le craindre et le combattre »

Apprentissage continu

« Agile development are built for learning »

Question :

Comment cela est-il mis en œuvre ?

- Implication du client -> feedback
- Review
- Retrospective

Implication du client

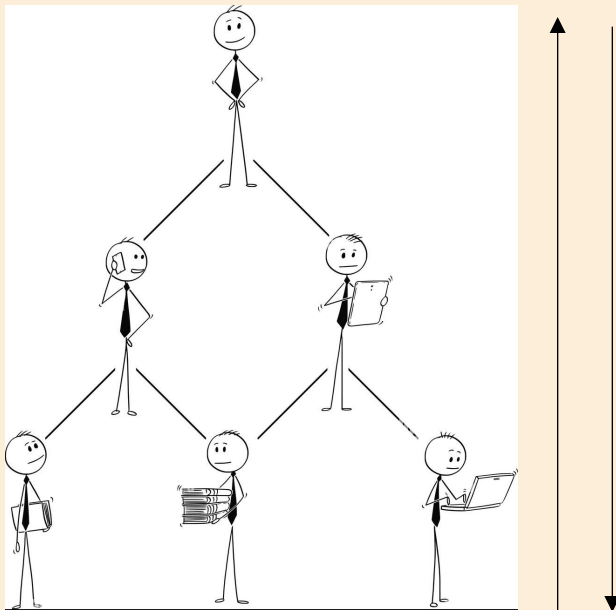
- Le produit répond toujours aux attentes de l'utilisateur et des clients
- De minimiser le coût d'un changement de direction, voire de l'anticiper
- Le client à une visibilité
- $\Rightarrow$ On évite ainsi le gaspillage d'argent

FEEDBACK

L'équipe propriétaire du processus

Question :

A votre avis, que cela peut-il signifier et impliquer

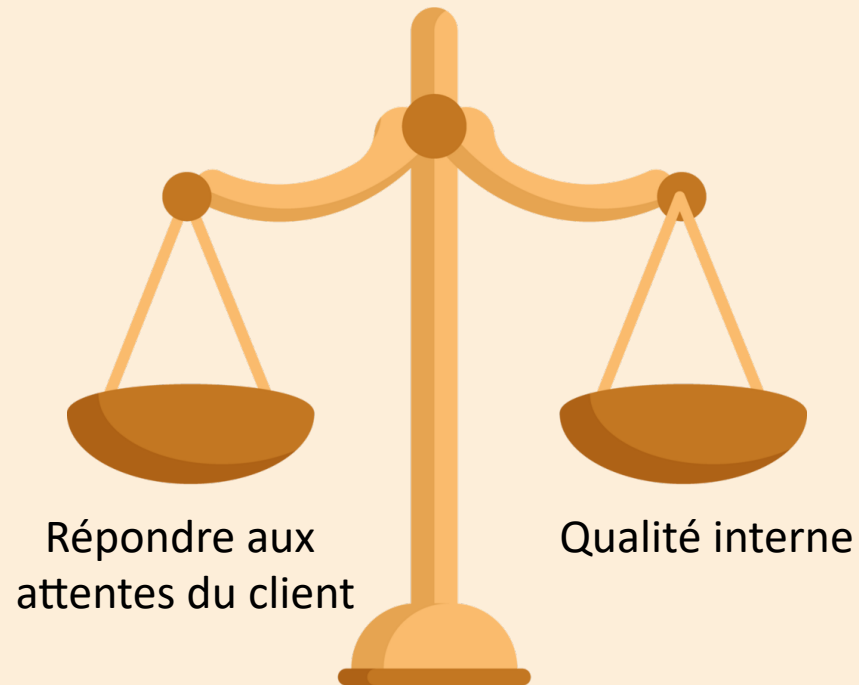


« Dans une approche d'amélioration continue, l'équipe est propriétaire de SON processus »

Un produit de haute qualité

Question :

C'est quoi un produit de haut qualité ?



Viser l'excellence

Affirmation

Agile est un engagement à tendre vers l'excellence, à être toujours plus professionnel et à favoriser les comportements professionnels dans l'industrie logiciel

Facilitateur Agile

Question :

Chef vs Facilitateur ?

Affirmation

Le Chef de Projet n'a plus le rôle autoritaire et de celui qui va assigner les tâches.

L'agilité préconise **l'auto-organisation**, ainsi le rôle du manager est de s'occuper de l'humain (montée en compétence, bien-être des salariés, etc).

Il a pour objectif de faire évoluer l'organisation dans le bon sens (amélioration continue des process).

⇒ Il donne un cadre de travail

Comment une équipe Agile travaille

Une équipe avec un but commun

- Communication importante entre toutes les équipes
 - Designer, architecte, codeur, testeurs travaillent ensemble pour ne former qu'une équipe
- Plusieurs rôles
 - Product owner
 - Client / stakeholders
 - Développeurs
 - Project Manager

Une équipe pluridisciplinaire



Question :

Qu'est une organisation en silo ?
Est-ce que ça fonctionne ?

- En évitant une organisation en silo, les équipes se rendent indépendantes
- Minimise les sollicitation grâce à un fort spectre de connaissances

Une équipe auto-organisée

- Ne signifie pas que les salariés conçoivent un modèle d'organisation
- Ne signifie pas non plus que les salariés font ce qu'ils veulent

Question :

Alors c'est quoi une équipe auto-organisée ?

Une équipe auto-organisée

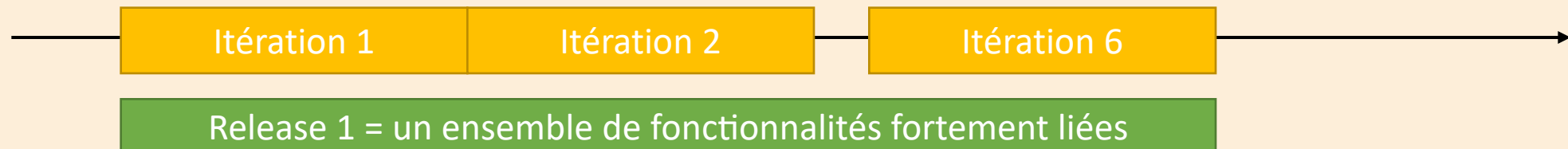
- Auto-organisation : signifie que la direction s'engage à accompagner les comportements émergents au lieu de spécifier un comportement efficace
- Besoin d'autonomie : levier essentiel à la motivation; au lieu d'adhérer à une politique, des protocoles, les équipes disposent d'un certain degres d'autonomie
 - Comment recruter un nouveau membre
 - Comment bien former les personnes
 - Quels sont les outils nécessaires
 - ...

Itérations courtes

- Les itérations sont timeboxées

Délivrer à chaque itération

- Un ou plusieurs requirements sont codés et potentiellement livrables
- Définition of DONE (nous y reviendrons avec Scrum)
- Comme une itération ne permet pas de produire assez de fonctionnalité pour satisfaire l'utilisateur le terme « release » a été introduit



Focus sur la valeur métier

- On priorise les fonctionnalités qu'on regroupe en « release »
 - Client + PO
- Pour exprimer ceci on peut écrire des US (nous y reviendrons dans un cours dédié)

Inspecter et adapter

- Un plan créé en début de projet n'a aucune garantie d'être tenu
- A chaque itération nous apprenons et adapte le plan en conséquence
 - Travailler ensemble
 - Les techno
 - Les attentes des utilisateurs
- Nous devenons de « plus en plus précis »

Pratiques Agile

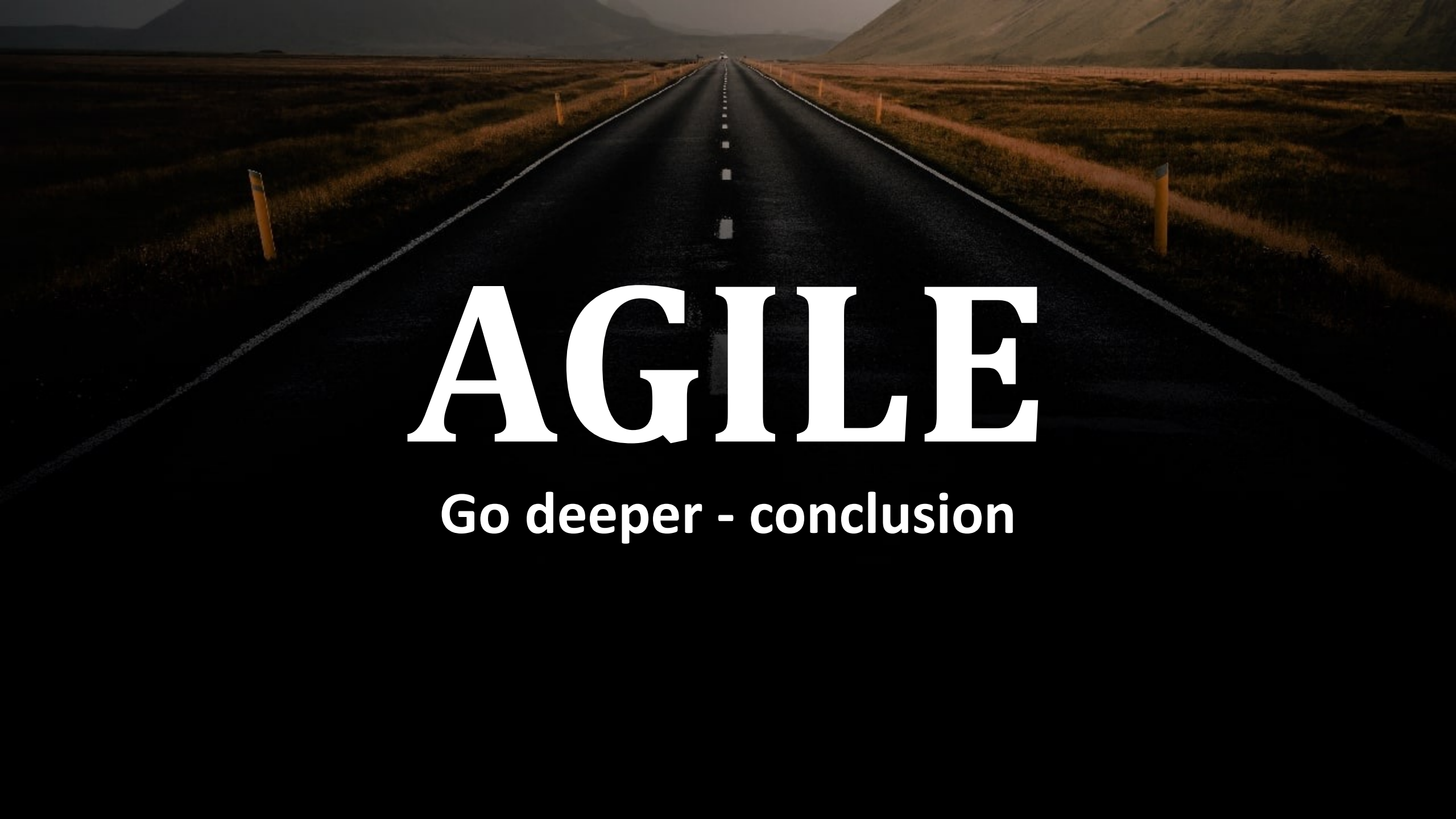
The Agile Subway Map



The colored “subway” lines represent practices from the various Agile approaches or areas of concern.

Click on the “station” circles to read the full definition for each term.



A long, straight road stretches into the distance under a dark sky, with hills in the background. The road is flanked by grassy fields and yellow posts. The word "AGILE" is written in large, white, serif capital letters across the center of the road.

AGILE

Go deeper - conclusion

MINDMAP